



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms- Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - <u>Network Flows</u> : Flow Networks, <u>Maximum Flows</u> : <u>Ford-Fulkerson</u> , Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	Contemporary Issues	2 hours



BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos ,Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K.Ahuja, Thomas L.Magnanti and James B. Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



Module 4: Graph Algorithms(6 Hours)

- All Pair Shortest Algorithm
 - Bellman Ford Algorithm
 - Floyd - Warshall Algorithm
- **Network Flows – Flow Networks**
- **Maximum Flow**
 - **Ford Fulkerson**
 - Edmond Karp
 - Push Re-label Algorithm



Flow Networks

- **Graph Applications:** Modelling a road map as a directed graph in order to find the shortest path from one point to another.
- Interpret a directed graph as a **“flow network”** and use it to answer questions about material flows.
- Imagine a material coursing through a system from a **source**, where the material is produced, to a **sink**, where it is consumed.
- Source produces the material at some steady rate, and the sink consumes the material at the same rate.
- **“Flow”** of the material at any point in the system is intuitively the rate at which the material moves.
- Flow networks can model many problems, including **liquids flowing through pipes**, **parts through assembly lines**, **current through electrical networks**, and **information through communication networks**.



Maximum Flow Problem-Idea

- The idea is to compute the greatest rate at which we can ship material from the source to the sink without violating any capacity constraints.
- It is one of the simplest problems concerning flow networks and this problem can be solved by efficient algorithms.
 - **Ford Fulkerson**
 - **Edmond Karp**
 - **Push Re-label Algorithm**



Flow Networks

- A flow network $G = (V, E)$ is a directed graph in which each edge $(u, v) \in E$ has a nonnegative capacity $C(u, v) \geq 0$.
- Further requires that if E contains an edge (u, v) , then there is no edge (v, u) in the reverse direction.
- If $(u, v) \notin E$, then for convenience we define $C(u, v) = 0$ and we disallow self-loops.
- We distinguish two vertices in a flow network: **a source s and a sink t** .
- For convenience, we **assume that each vertex lies on some path from the source to the sink**. i.e, for each vertex $v \in V$, the flow network contains a path $s \rightarrow v \rightarrow t$.
- The graph is therefore connected and, since each vertex **other than s has at least one entering edge**.



Flow Networks – Formal Definition

- Let $G = (V, E)$ be a flow network with a capacity function “**C**”. Let **S** be the source of the network, and let **t** be the sink. A flow in G is a real-valued function **f** : **V*****V**->**R** that satisfies the following two properties:

- Capacity Constraint:**

For all $u, v \in V$, we require $0 \leq f(u, v) \leq c(u, v)$.

- Flow Conservation:**

For all $u, v \in V - \{s, t\}$

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v) .$$

When $(u, v) \notin E$, there can be no flow from u to v , and $f(u, v) = 0$.



Maximum Flow

- In a given flow network G with source s and sink t , objective is to find a flow of maximum value.

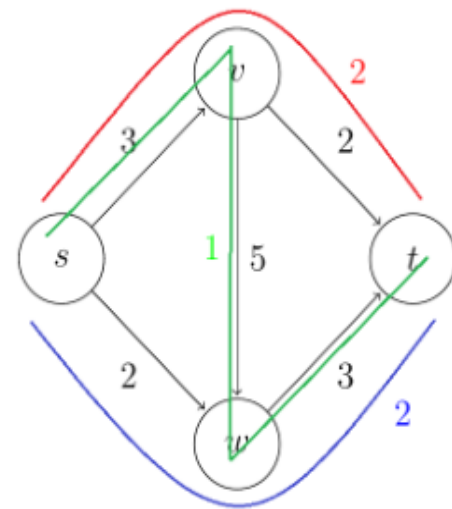
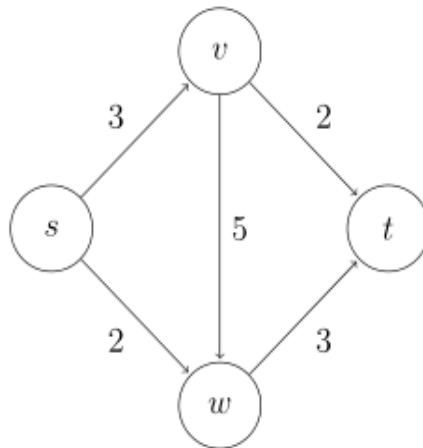


Figure 1: Our first flow network. Each edge is associated with a capacity.

Figure 2: A sample flow of value 5, where the red, green and blue paths have flow values of 2, 1, 2 respectively.

Maximum Flow – Ford-Fulkerson method

- **Depends on three important concepts:**

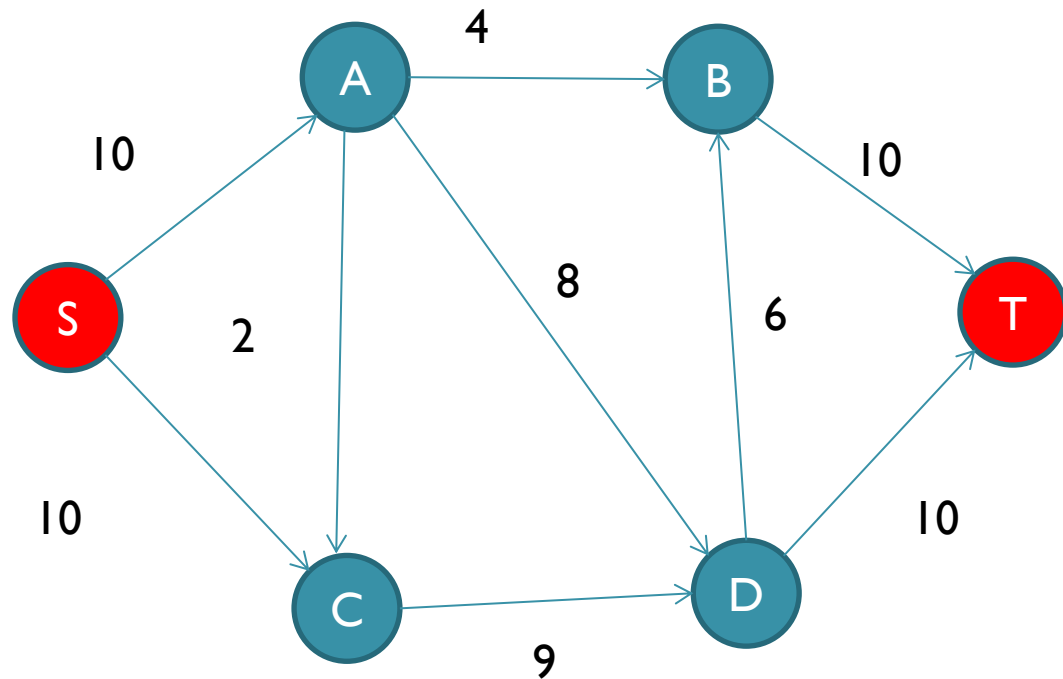
1. Residual Networks 2. Augmenting Path 3. Cut

- **Residual Graph** - It is a graph is nothing but the same graph with the only difference is we use **residual capacities as capacities**. It indicates additional possible flow, if there is such path from source to sink, then there is a possibility to add flow.
- **Residual Capacity** - Residual capacity of the directed edge is nothing but **capacity of the edge - current flow through the edge**. If there is flow along a directed edge $u \rightarrow v$ then reversed edge has a capacity 0 and we can say $f(v,u) = -f(u,v)$
- **Augmenting Path** - It is a simple **path from source to sink** in a residual graph i.e. along the edges whose capacity is 0.
 - Non- Full forward edges
 - Non-empty(zero) backward edges
- **Minimal Cut- Bottneck capacity**, which decides maximum possible flow from source to sink through augment path.



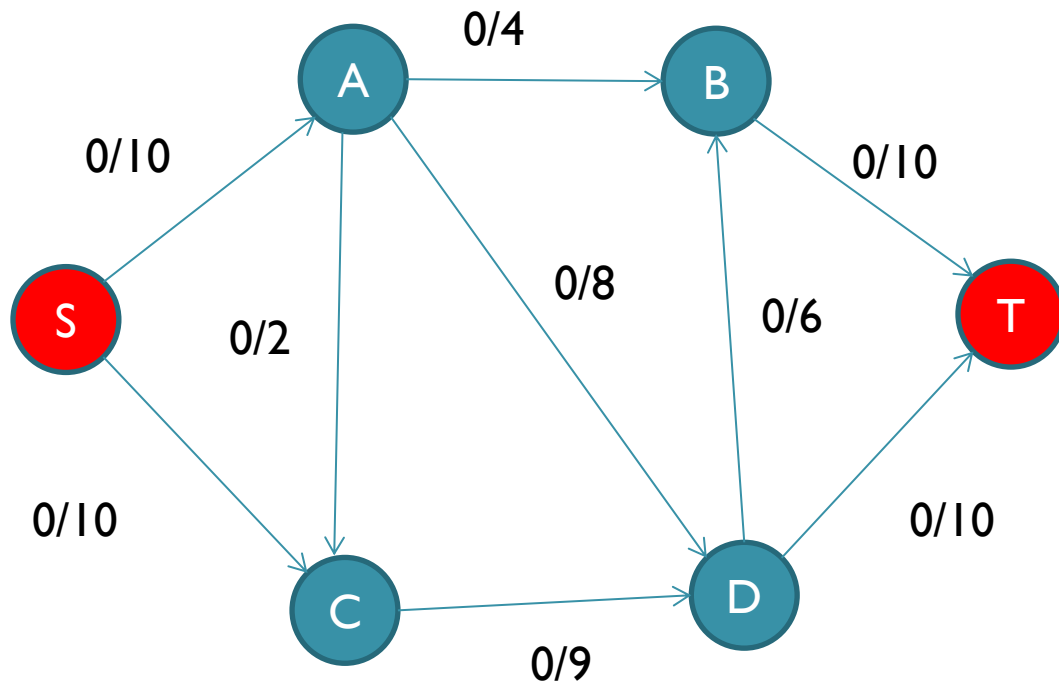


Maximum Flow – Ford-Fulkerson method



Source: S
Sink: T

Initialize flow f to 0



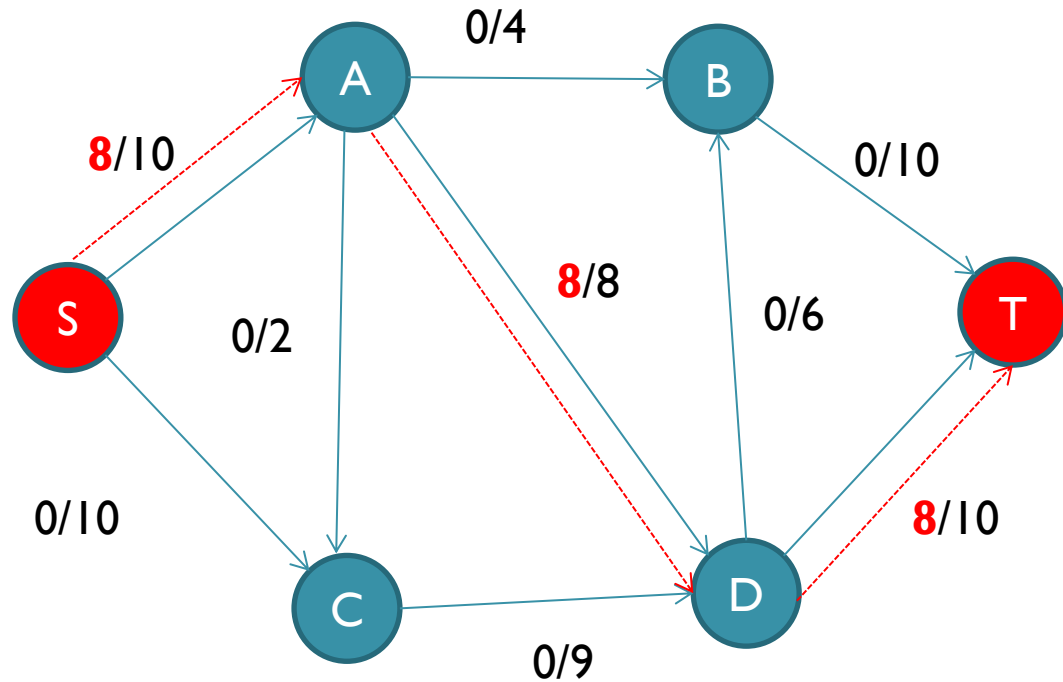
Source: S
Sink: T



Consider Augmenting path I

Source: S

Sink: T

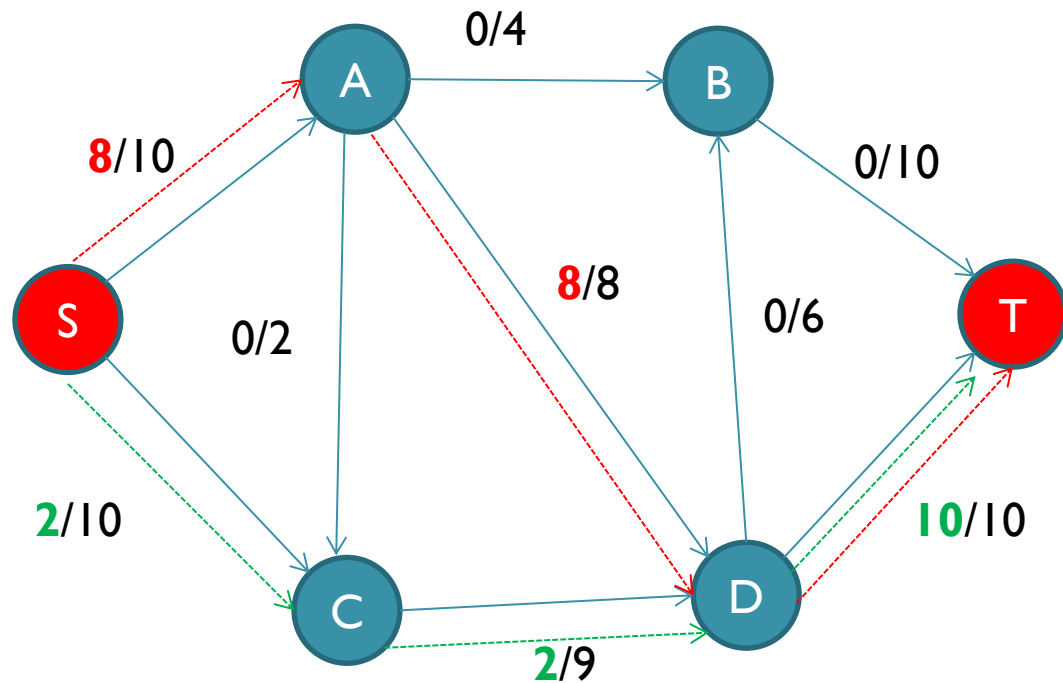


Augmenting Path	Bottle neck Capacity
S-A-D-T	8

Consider Augmenting path 2

Source: S

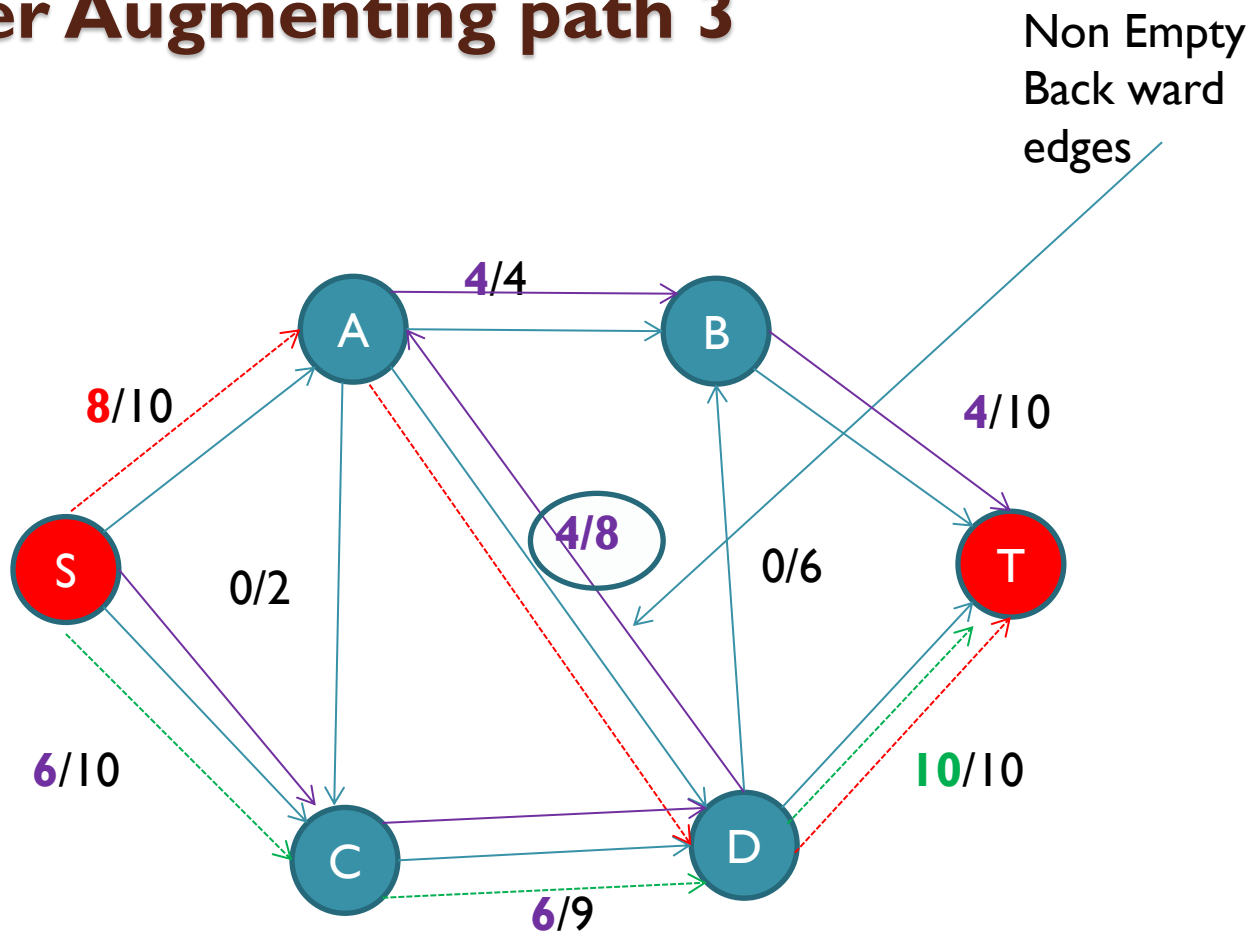
Sink: T



Augmenting Path	Bottle neck Capacity
S-A-D-T	8
S-C-D-T	2

Consider Augmenting path 3

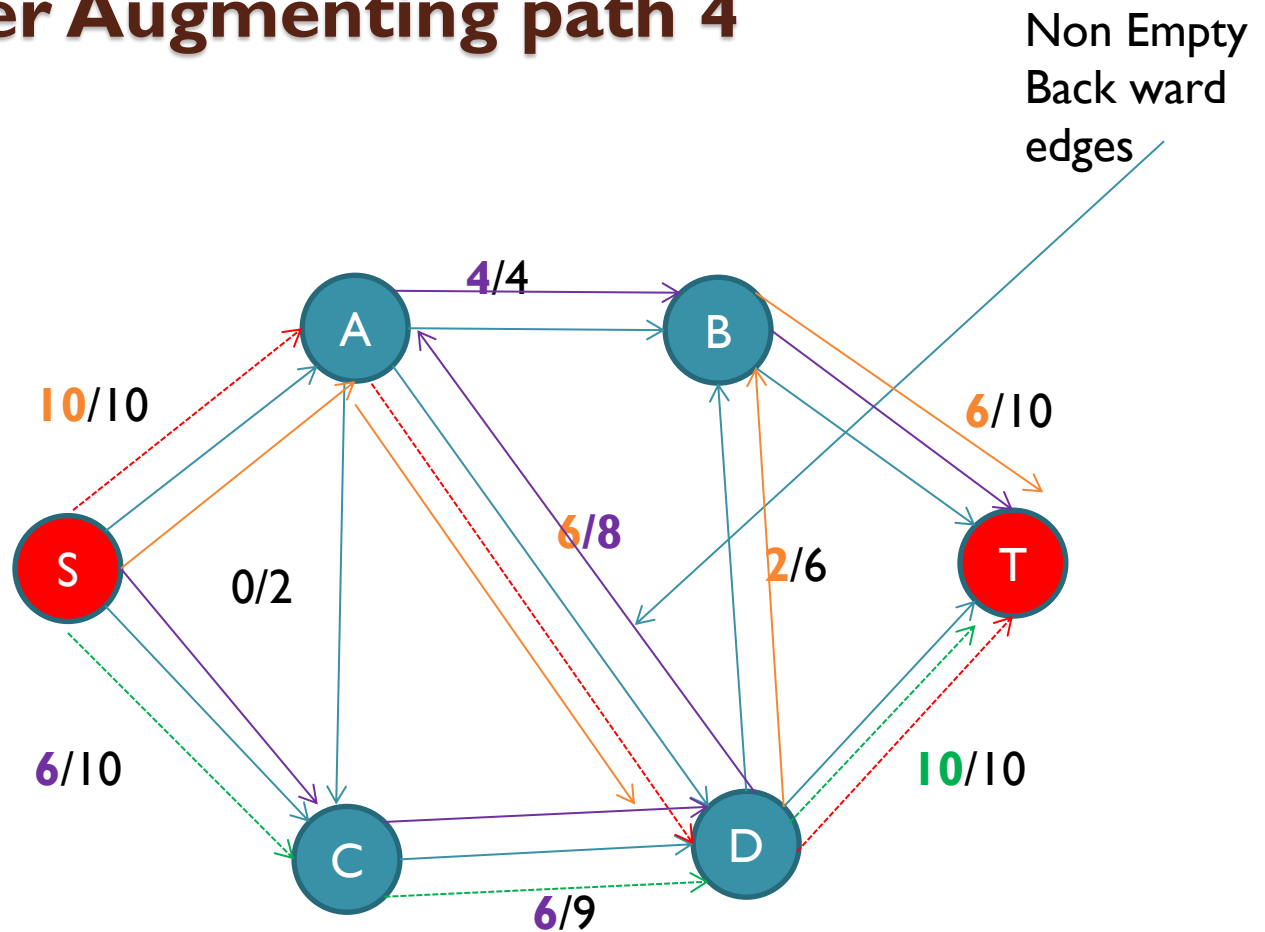
Source: S
Sink: T



Augmenting Path	Bottle neck Capacity
S-A-D-T	8
S-C-D-T	2
S-C-D-A-B-T	4

Consider Augmenting path 4

Source: S
Sink: T

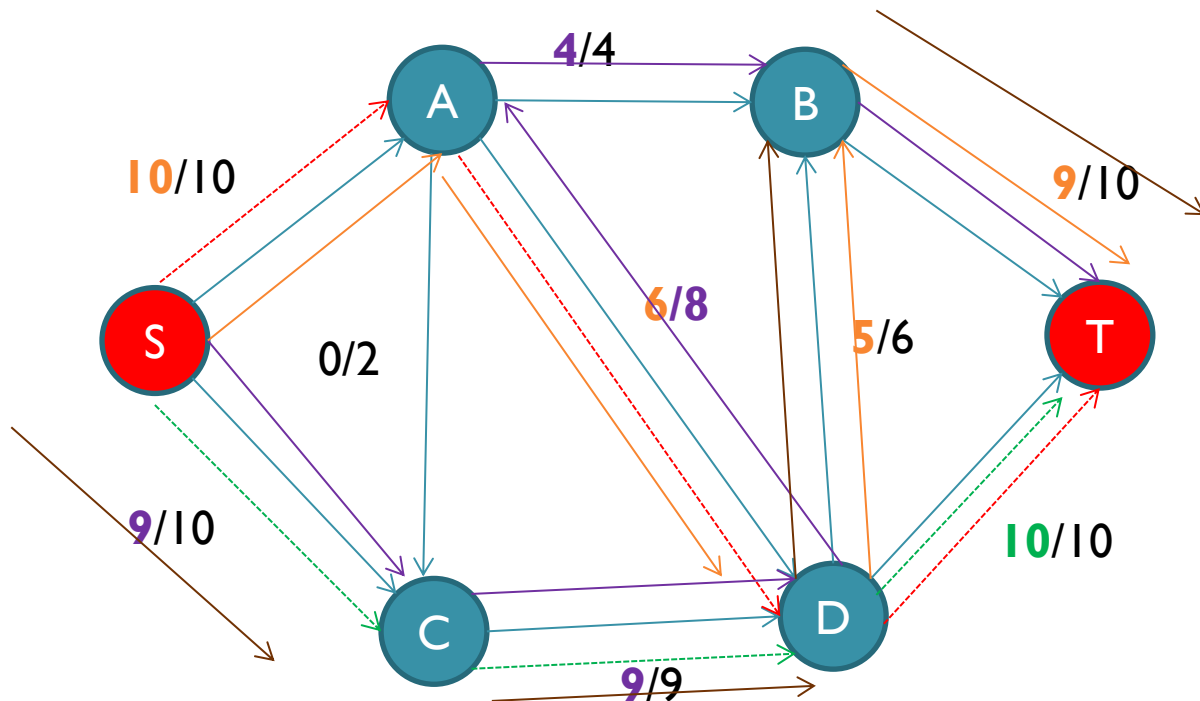


Augmenting Path	Bottle neck Capacity
S-A-D-T	8
S-C-D-T	2
S-C-D-A-B-T	4
S-A-D-B-T	2

Consider Augmenting path 5

Source: S

Sink: T



Augmenting Path	Bottle neck Capacity
S-A-D-T	8
S-C-D-T	2
S-C-D-A-B-T	4
S-A-D-B-T	2
S-C-D-B-T	3

Maximum Flow – Ford-Fulkerson method

FORD-FULKERSON-METHOD(G, s, t)

- 1 initialize flow f to 0
- 2 **while** there exists an augmenting path p in the residual network G_f
- 3 augment flow f along p
- 4 **return** f

FORD-FULKERSON(G, s, t)

- 1 **for** each edge $(u, v) \in G.E$
- 2 $(u, v).f = 0$
- 3 **while** there exists a path p from s to t in the residual network G_f
- 4 $c_f(p) = \min \{c_f(u, v) : (u, v) \text{ is in } p\}$
- 5 **for** each edge (u, v) in p
- 6 **if** $(u, v) \in E$
- 7 $(u, v).f = (u, v).f + c_f(p)$
- 8 **else** $(v, u).f = (v, u).f - c_f(p)$



Ford-Fulkerson method – Analysis

1. Finding an Augmenting path: $O(E)$
E- represents set of edges
2. Compute Bottle neck capacity- max flow in each path
3. Augment each edge and update the total flow. $O(F)$

Overall analysis : $O(E * F)$



Ford-Fulkerson method – Example

